Ausfüllbare PDF-Formulare mit LaTeX erstellen

Fortbildungsmaterial für MINT-Lehrkräfte

11. November 2026

Ein Arbeitsblatt oder eine Leistungskontrolle, die Schülerinnen und Schüler direkt am Tablet oder Laptop ausfüllen, speichern und über die Lernplattform zurückgeben: Dafür braucht man kein Zusatzprogramm. LaTeX kann mit dem Paket `hyperref` echte PDF-Formularfelder erzeugen, also Textfelder, Ankreuzkästchen, Optionsfelder und Auswahllisten. Diese Handreichung zeigt Schritt für Schritt, wie das geht, und baut am Ende eine vorhandene Papier-Leistungskontrolle zu einer digitalen Fassung um. Ausgewertet wird dabei nichts automatisch: Die Lehrkraft liest jede Abgabe selbst, wie bei Papier auch. Die Felder in den blauen Ausgabe-Kästen dieser Handreichung sind echt. Sie können sie im PDF direkt ausprobieren.

1 Grundlagen und benötigte Pakete

1.1 Was man braucht

Formularfelder gehören zum Paket `hyperref`. Es ist in der universalen `header.tex` bereits geladen. Zwei Dinge kommen für ein Formular dazu:

- Alle Felder stehen innerhalb der Umgebung `Form`. Am einfachsten umschließt sie den ganzen Inhalt, direkt nach `\begin{document}` bis direkt vor `\end{document}`.
- In der Präambel steht `\hypersetup{unicode=false}`. Das ist nur für Optionsfelder (Kapitel 3.3) nötig, schadet aber den anderen Feldern nicht. Ohne diese Zeile erzeugt `hyperref` (getestet mit Version 7.01h) für Optionsfelder unlesbare interne Namen.

Das Grundgerüst eines Formulars sieht damit so aus:

```
\input{header.tex}
\input{newcommands.tex}
\hypersetup{unicode=false}

\begin{document}
\begin{Form}

  ... Aufgaben mit Feldern ...

\end{Form}
\end{document}
```

Kompiliert wird wie immer mit `pdflatex`. JavaScript oder besondere Programme sind nicht nötig.

1.2 Die Feldtypen im Überblick

Befehl	Wofür
`\TextField`	Textfeld, ein- oder mehrzeilig
`\CheckBox`	Ankreuzkästchen, unabhängig von anderen
`\ChoiceMenu[radio]`	Optionsfelder: genau eine Antwort aus mehreren
`\ChoiceMenu [combo, popdown]`	Auswahlliste zum Aufklappen
`\Reset`	Knopf, der alle Felder leert (für Leistungskontrollen ungeeignet, siehe Kapitel 8)

Für den Schulalltag stehen in der universalen `newcommands.tex` vier Kurzbefehle bereit, die auf diesen Feldern aufbauen: `\feld`, `\zeichen`, `\kreuz` und `\antwort`. Sie werden in den folgenden Kapiteln vorgestellt.

2 Textfelder

2.1 Ein erstes Textfeld

Jedes Feld braucht einen **Namen**. Unter diesem Namen speichert das PDF später die Eingabe. Die leeren geschweiften Klammern am Ende sind die Beschriftung, die `hyperref` vor das Feld setzen würde. Wir lassen sie leer und schreiben die Beschriftung selbst davor:

```
Name: \TextField[name=bsp-name, width=6cm]{}
```

Ausgabe

Name:

Die Breite legt `width` fest. Ohne Angabe ist das Feld 3 cm breit. Der blaue Rand und der hellblaue Hintergrund kommen aus der `newcommands.tex` (Kapitel 4), ebenso die Schriftgröße 11 pt für das, was eingetippt wird.

2.2 Größe, Schrift und Vorbelegung

Mit `height` wird das Feld höher, mit `charsize` die Schrift größer. `value` trägt einen Text vor, den man überschreiben kann, zum Beispiel die Klasse:

```
Klasse: \TextField[name=bsp-klasse, width=2cm, value={10b}]{} \qquad
Ort: \TextField[name=bsp-ort, width=5cm, height=0.9cm, charsize=16pt]{}
```

Ausgabe

Klasse: 10b Ort:

Hier sieht man auch einen Schönheitsfehler: Ein Feld sitzt mit seiner Unterkante auf der Grundlinie und ragt deshalb über den Text hinaus. Im nächsten Abschnitt wird das gelöst.

2.3 Lücken im Satz: `\feld`

Der Kurzbefehl `\feld[Breite]{Name}` setzt ein Textfeld etwas tiefer, sodass der eingetippte Text auf derselben Linie steht wie der Satz. Er ersetzt die Schreiblücke eines Papierblatts:

```
Der Operator \texttt{//} heißt \feld[4cm]{bsp-op1}, der Operator
\texttt{\%} heißt \feld[4cm]{bsp-op2}.
```

Ausgabe

Der Operator `//` heißt , der Operator `%` heißt .

Ohne Breitenangabe ist das Feld 2,4 cm breit. Wichtig ist der Name: Er muss im ganzen Dokument eindeutig sein. Mehr dazu in Kapitel 4.3.

2.4 Genau ein Zeichen: `\zeichen`

Für Zuordnungen mit einem Buchstaben (E, V oder A; die Reihenfolge von Programmzeilen) gibt es `\zeichen{Name}`. Das Feld nimmt nur ein Zeichen an (`maxlen=1`) und zeigt es zentriert (`align=1`):

```
Reihenfolge: \zeichen{bsp-r1}\ \zeichen{bsp-r2}\ \zeichen{bsp-r3}\
\zeichen{bsp-r4}
```

Ausgabe

Reihenfolge:

2.5 Mehrzeilige Antworten: `\antwort`

Wo auf Papier Schreiblinien stehen, braucht man ein mehrzeiliges Feld. Der Grundbefehl ist ein `\TextField` mit `multiline=true`:

```
\TextField[name=bsp-frei, multiline=true, width=12cm, height=2cm]{}
```

Ausgabe

Bequemer ist `\antwort[Höhe]{Name}`. Das Feld beginnt in einer neuen Zeile, nimmt die ganze Zeilenbreite ein und ist ohne Angabe 3,2 cm hoch. Das entspricht etwa vier Schreiblinien und bietet bei 11 pt Schrift Platz für sechs bis sieben getippte Zeilen:

```
Erkläre den Unterschied zwischen einem Syntaxfehler und einem
Semantikfehler.
\antwort[2.5cm]{bsp-fehler}
```

Ausgabe

Erkläre den Unterschied zwischen einem Syntaxfehler und einem Semantikfehler.

Zeilenumbrüche mit der Eingabetaste sind in mehrzeiligen Feldern erlaubt. Wird mehr getippt, als hineinpasst, scrollt das Feld. Beim Ausdrucken fehlt der überstehende Text dann allerdings, deshalb die Felder lieber großzügig planen.

3 Ankreuzen und Auswählen

3.1 Ankreuzkästchen: `\kreuz`

`\CheckBox` erzeugt ein Kästchen, das per Klick ein Häkchen bekommt. `\kreuz{Name}` ist dasselbe in passender Größe und Höhe für den Fließtext:

```
Kreuze an, welche Zeile zur Verarbeitung gehört.\\
\kreuz{bsp-z1}\ Zeile 1 \qquad \kreuz{bsp-z2}\ Zeile 2 \qquad
\kreuz{bsp-z3}\ Zeile 3
```

Ausgabe

Kreuze an, welche Zeile zur Verarbeitung gehört.

☐ Zeile 1 ☐ Zeile 2 ☐ Zeile 3

Jedes Kästchen ist ein eigenes Feld. Man kann also auch mehrere ankreuzen, so wie auf Papier. Für Aufgaben wie „Kreuze alle richtigen Aussagen an“ ist das genau richtig.

3.2 Kästchen in Tabellen

Richtig-falsch-Tabellen und Zuordnungen funktionieren genauso, das Kästchen steht einfach in der Tabellenzelle:

```
\begin{tabular}{p{7.5cm}cc}
  \toprule
  Aussage & richtig & falsch \\
  \midrule
  \texttt{"7"} ist eine Zahl. & \kreuz{bsp-k1r} & \kreuz{bsp-k1f} \\
  \texttt{2.0} ist ein \texttt{float}. & \kreuz{bsp-k2r} & \kreuz{bsp-k2f} \\
  \bottomrule
\end{tabular}
```

Ausgabe

Aussage	richtig	falsch
`"7"` ist eine Zahl.	☐	☐
`2.0` ist ein `float`.	☐	☐

Dass das so ordentlich aussieht, liegt an einer Kleinigkeit in `\kreuz`: Das Kästchen steckt in einer eigenen Box (`\raisebox`). Ein nacktes `\CheckBox` übernimmt in einer Tabelle die Höhe der ganzen Zeile und wird zu einem langen, schmalen Rechteck. Dasselbe gilt für Textfelder. Alle vier Kurzbefehle sind deshalb so gebaut.

3.3 Optionsfelder: genau eine Antwort

Soll nur eine Antwort möglich sein, nimmt man Optionsfelder. Klickt man ein anderes an, verschwindet die erste Markierung. Alle Optionen gehören zu einem `\ChoiceMenu` mit dem Schlüssel `radio`. Die Liste am Ende hat die Form `Beschriftung=Wert`:

```
\texttt{2.0} hat den Datentyp
\ChoiceMenu[radio, name=bsp-typ, default=]{}{int=int,
  float=float, str=str}
```

Ausgabe

2.0 hat den Datentyp int ☐ float ☐ str ☐

Die Beschriftung steht dabei immer vor dem Knopf. `default=` (leer) sorgt dafür, dass anfangs nichts ausgewählt ist. Der Wert hinter dem Gleichheitszeichen ist das, was später gespeichert wird.

Optionsfelder brauchen `\hypersetup{unicode=false}` in der Präambel (siehe Kapitel 1.1). Und sie stehen immer in einer Reihe nebeneinander. Für Tabellen mit Spalten „richtig“ und „falsch“ nimmt man deshalb besser Kästchen wie in Kapitel 3.2.

3.4 Auswahllisten

Eine Auswahlliste klappt beim Anklicken auf. Sie spart Platz und verhindert Tippfehler, zum Beispiel bei E, V oder A:

```
Der Automat berechnet das Wechselgeld.
\ChoiceMenu[combo, popdown, name=bsp-eva, width=1.6cm]{}{E,V,A}
```

Ausgabe

Der Automat berechnet das Wechselgeld. ☐▾

Es braucht beide Schlüssel: `popdown` allein erzeugt eine fest aufgeklappte Liste, `combo` allein ein Feld, in das man auch frei tippen kann. Anfangs ist das Feld leer, eine Antwort entsteht erst durch die Auswahl (getestet in Chrome). Auf Tablets sind Auswahllisten etwas umständlicher zu bedienen als ein Feld für ein Zeichen. Für Leistungskontrollen ist `\zeichen` deshalb die robustere Wahl.

4 Einheitliches Aussehen und Feldnamen

4.1 Standardwerte für alle Felder

`hyperref` liest das Aussehen jedes Feldtyps aus einem `\DefaultOptionsof...`-Befehl. In der `newcommands.tex` sind diese einmal für alle Dokumente festgelegt:

```
\renewcommand{\DefaultOptionsofText}{print, bordercolor={0 0.353 0.612},
  backgroundcolor={0.94 0.96 1}, charsize=11pt}
\renewcommand{\DefaultOptionsofCheckBox}{print, bordercolor={0 0.353 0.612},
  backgroundcolor={0.94 0.96 1}, charsize=11pt}
\renewcommand{\DefaultHeightofText}{0.6cm}
```

Farben stehen hier nicht als Farbname, sondern als drei Zahlen zwischen 0 und 1 für Rot, Grün und Blau. `print` sorgt dafür, dass die Felder samt Inhalt mitgedruckt werden. Wer ein Feld einmal anders haben möchte, gibt die Option direkt beim Feld an, sie gilt dann nur dort:

```
\TextField[name=bsp-rot, width=4cm, bordercolor={0.8 0 0},
  backgroundcolor={1 0.94 0.94}]{}
```

Ausgabe

4.2 Die vier Kurzbefehle

So sind die Kurzbefehle in der `newcommands.tex` definiert:

```
\newcommand{\feld}[2][2.4cm]{\raisebox{-0.17cm}{\TextField[name=#2, width=#1]{}}}
\newcommand{\zeichen}[1]{\raisebox{-0.12cm}{\TextField[name=#1, width=0.8cm,
  height=0.5cm, maxlen=1, align=1]{}}}
\newcommand{\kreuz}[1]{\raisebox{-0.08cm}{\CheckBox[name=#1, width=0.42cm,
  height=0.42cm]{}}}
\newcommand{\antwort}[2][3.2cm]{\par\vspace{0.15cm}\noindent
  \hbox{\TextField[name=#2, multiline=true, width=\linewidth, height=#1]{}}}
```

4.3 Feldnamen sinnvoll vergeben

Der Name eines Feldes taucht für die Schülerinnen und Schüler nirgends auf. Für die Lehrkraft ist er aber wichtig, sobald man Antworten einsammelt (Kapitel 7). Bewährt hat sich ein Schema nach Aufgabennummer:

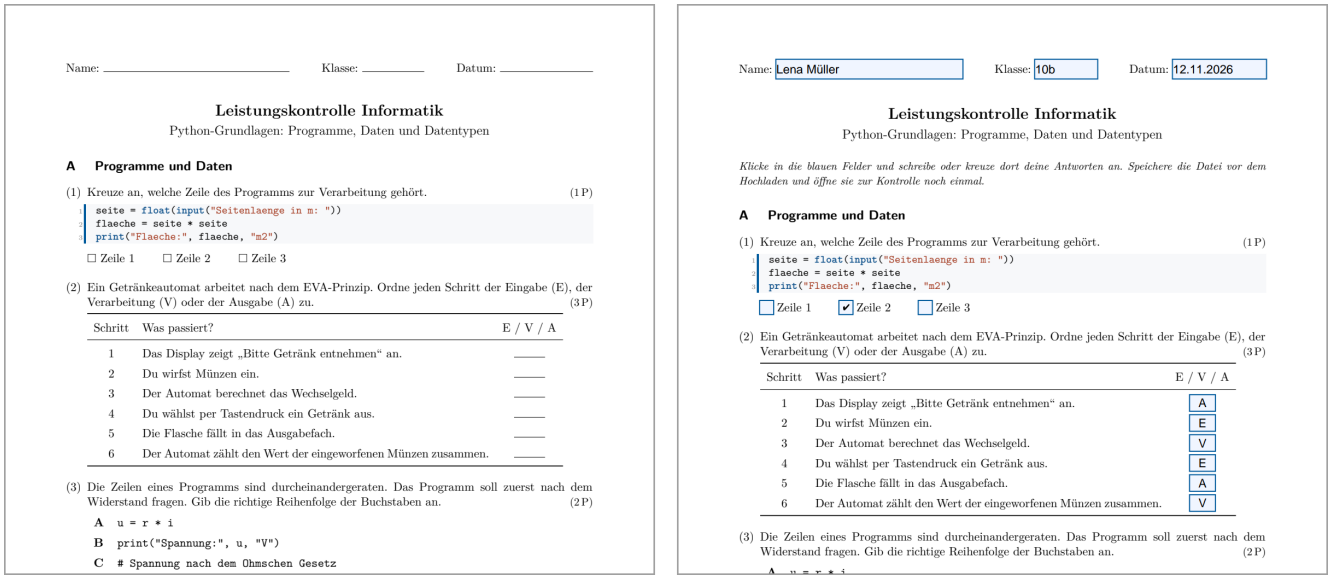
`name`, `klasse`, `datum`	Kopf des Blattes
`a2-s3`	Aufgabe 2, Schritt 3
`a4-1-r`, `a4-1-f`	Aufgabe 4, Aussage 1, Spalte richtig/falsch
`a5`	Aufgabe 5, freie Antwort
`z-1`	Zusatzaufgabe, Lücke 1

Zwei Regeln dazu, beide getestet:

- **Bindestrich statt Unterstrich.** Ein Unterstrich im Namen wird von `hyperref` stillschweigend entfernt: Aus `a4_1_r` wird `a41r`.
- **Jeder Name nur einmal.** Zwei Felder mit demselben Namen behandelt nicht jedes Programm gleich. In Chrome sind es zwei getrennte Felder, andere Programme können sie zusammenlegen.

5 Praxis: eine Leistungskontrolle digital umbauen

Als Beispiel dient eine Leistungskontrolle für Klasse 10 (Python-Grundlagen, 8 Aufgaben, 20 Punkte), die es bereits als Papierfassung gab. Die digitale Fassung sieht gleich aus. Nur dort, wo auf Papier geschrieben oder angekreuzt wird, steht jetzt ein Feld.



Name: _____ Klasse: _____ Datum: _____

Leistungskontrolle Informatik

Python-Grundlagen: Programme, Daten und Datentypen

A Programme und Daten

(1) Kreuze an, welche Zeile des Programms zur Verarbeitung gehört. (1 P)

```
seite = float(input("Seitenlaenge in m: "))
flaeche = seite * seite
print("Flaeche:", flaeche, "m2")
```

☐ Zeile 1 ☐ Zeile 2 ☐ Zeile 3

(2) Ein Getränkeautomat arbeitet nach dem EVA-Prinzip. Ordne jeden Schritt der Eingabe (E), der Verarbeitung (V) oder der Ausgabe (A) zu. (3 P)

Schritt	Was passiert?	E / V / A
1	Das Display zeigt „Bitte Getränk entnehmen“ an.	____
2	Du wirfst Münzen ein.	____
3	Der Automat berechnet das Wechselgeld.	____
4	Du wählst per Tastendruck ein Getränk aus.	____
5	Die Flasche fällt in das Ausgabefach.	____
6	Der Automat zählt den Wert der eingeworfenen Münzen zusammen.	____

(3) Die Zeilen eines Programms sind durcheinandergeraten. Das Programm soll zuerst nach dem Widerstand fragen. Gib die richtige Reihenfolge der Buchstaben an. (2 P)

```
A  u = r * i
B  print("Spannung:", u, "V")
C  # Spannung nach dem Ohmschen Gesetz
```

Name: Lena Müller Klasse: 10b Datum: 12.11.2026

Leistungskontrolle Informatik

Python-Grundlagen: Programme, Daten und Datentypen

Klicke in die blauen Felder und schreibe oder kreuze dort deine Antworten an. Speichere die Datei vor dem Hochladen und öffne sie zur Kontrolle noch einmal.

A Programme und Daten

(1) Kreuze an, welche Zeile des Programms zur Verarbeitung gehört. (1 P)

```
seite = float(input("Seitenlaenge in m: "))
flaeche = seite * seite
print("Flaeche:", flaeche, "m2")
```

☐ Zeile 1 ☑ Zeile 2 ☐ Zeile 3

(2) Ein Getränkeautomat arbeitet nach dem EVA-Prinzip. Ordne jeden Schritt der Eingabe (E), der Verarbeitung (V) oder der Ausgabe (A) zu. (3 P)

Schritt	Was passiert?	E / V / A
1	Das Display zeigt „Bitte Getränk entnehmen“ an.	A
2	Du wirfst Münzen ein.	E
3	Der Automat berechnet das Wechselgeld.	V
4	Du wählst per Tastendruck ein Getränk aus.	E
5	Die Flasche fällt in das Ausgabefach.	A
6	Der Automat zählt den Wert der eingeworfenen Münzen zusammen.	V

(3) Die Zeilen eines Programms sind durcheinandergeraten. Das Programm soll zuerst nach dem Widerstand fragen. Gib die richtige Reihenfolge der Buchstaben an. (2 P)

Links die Papierfassung, rechts die digitale Fassung, ausgefüllt und gespeichert in Chrome.

5.1 Der Umbau in fünf Schritten

(1) **Lösungsteil herausnehmen.** Er kommt in eine eigene Datei für die Lehrkraft. Alles, was im PDF steht, kann man auslesen, auch unsichtbar gemachten Text.

(2) **`Form` um den Inhalt legen** und `\hypersetup{unicode=false}` ergänzen.

(3) **Ersetzen:** Kästchen durch `\kreuz`, Schreiblücken durch `\feld`, einzelne Buchstaben durch `\zeichen`, Schreiblinien durch `\antwort`.

(4) **Kopf:** Name, Klasse und Datum als Felder in den Text setzen. Die Kopfzeilen-Befehle wie `\lk{10b}{Thema}` setzen Name und Punkte nur als Text in die Kopfzeile, dort gibt es keine Felder.

(5) **Hinweis für die Schülerinnen und Schüler** unter den Titel schreiben, wie bearbeitet und abgegeben wird.

5.2 Kopf und Hinweis

```
Name: \feld[5.5cm]{name} \hfill Klasse: \feld[1.8cm]{klasse}
\hfill Datum: \feld[2.6cm]{datum}

{\small\itshape Klicke in die blauen Felder und schreibe oder kreuze dort
deine Antworten an. Speichere die Datei vor dem Hochladen und öffne sie zur
Kontrolle noch einmal.}
```

Ausgabe

Name: Klasse: Datum:

Klicke in die blauen Felder und schreibe oder kreuze dort deine Antworten an. Speichere die Datei vor dem Hochladen und öffne sie zur Kontrolle noch einmal.

5.3 Eine Aufgabe vorher und nachher

Auf Papier stand in der Aufgabe zu den Datentypen in jeder Zelle ein leeres Kästchen `\kasten` (☐). In der digitalen Fassung bekommt jedes Kästchen einen eigenen Namen aus Aufgabe, Zeile und Spalte:

```
\begin{tabular}{>{\ttfamily}l ccc}
  \toprule
  \textrm{Wert} & \texttt{int} & \texttt{float} & \texttt{str} \\
  \midrule
  15 & \kreuz{a7-1-int} & \kreuz{a7-1-float} & \kreuz{a7-1-str} \\
  1.72 & \kreuz{a7-2-int} & \kreuz{a7-2-float} & \kreuz{a7-2-str} \\
  "04109" & \kreuz{a7-3-int} & \kreuz{a7-3-float} & \kreuz{a7-3-str} \\
  \bottomrule
\end{tabular}
```

Ausgabe

Wert	int	float	str
15	☐	☐	☐
1.72	☐	☐	☐
"04109"	☐	☐	☐

Eine Aufgabe mit Schreiblinien wird zu einem Antwortfeld:

```
\begin{enumerate}[(1)]
  \setcounter{enumi}{4}
  \item Nenne vier Regeln, die du beim Festlegen eines Variablennamens in
  Python beachten musst. \hfill (4\,P)
  \antwort{a5}
\end{enumerate}
```

Ausgabe

(5) Nenne vier Regeln, die du beim Festlegen eines Variablennamens in Python beachten musst. (4 P)

Dieser Handreichung liegen bei:

- `LK_Python_Grundlagen_digital.tex`: die vollständige Leistungskontrolle,
- `vorlage_lk_digital.tex`: eine kurze Vorlage zum Weiterbauen.

6 Austeilen, bearbeiten, einsammeln

6.1 Der Ablauf

(1) **Austeilen:** Die PDF-Datei (ohne Lösungen) über die Lernplattform bereitstellen, zum Beispiel als Datei in einer LernSax-Aufgabe.

(2) **Bearbeiten:** Die Schülerinnen und Schüler laden die Datei herunter, öffnen sie in einem Programm, das Formulare kann (siehe 6.2), und füllen die Felder aus.

(3) **Speichern und prüfen:** Die Datei speichern, schließen und noch einmal öffnen. Stehen die Antworten noch darin, hat das Speichern geklappt. Dieser Schritt ist der wichtigste, er kostet eine Minute und verhindert leere Abgaben.

(4) **Abgeben:** Die Datei mit dem eigenen Namen im Dateinamen hochladen, zum Beispiel `LK_Mueller_Lena.pdf`.

(5) **Bewerten:** Die Lehrkraft lädt die Abgaben herunter und korrigiert jede einzeln, zum Beispiel mit der Kommentar- oder Stiftfunktion des PDF-Programms, und gibt die korrigierte Datei zurück.

Für eine Leistungskontrolle gelten dieselben Regeln wie auf Papier: Bearbeitung im Raum, unter Aufsicht, in der festgelegten Zeit. Ob in dieser Zeit andere Programme oder das Internet offen sind, lässt sich am Gerät nicht erkennen. Das muss die Aufsicht regeln.

6.2 Welche Programme Formulare können

Die Felder sind gewöhnliche PDF-Formularfelder ohne JavaScript. Getestet wurde für diese Handreichung Folgendes:

Programm	**Ergebnis im Test**
Chrome und Edge (PDFium)	Eintippen, Ankreuzen, Optionsfelder und Auswahllisten funktionieren, Umlaute und ß werden gespeichert und richtig wieder angezeigt. Getestet durch eine Simulation, die klickt und tippt wie ein Mensch.
Poppler (z. B. Okular, Evince)	Text wird richtig angezeigt, ein Häkchen erscheint dort nur als schmaler Strich.

Adobe Acrobat Reader ist das Programm, für das PDF-Formulare ursprünglich gemacht sind. Apple Vorschau, die Dateien-App auf dem iPad und Notiz-Apps wie GoodNotes können ebenfalls Formularfelder ausfüllen, wurden für diese Handreichung aber nicht getestet. Deshalb lohnt sich vor dem ersten Einsatz eine **Probeabgabe**: In der Stunde davor ein kleines Formular mit einem Feld jeder Art ausfüllen, speichern und hochladen lassen. Dann weiß man, was auf den Geräten der Klasse funktioniert.

7 Antworten einsammeln mit Python (optional)

Wer nicht jede Datei einzeln öffnen möchte, kann die Antworten aller Abgaben in eine Tabelle holen. Das Skript `antworten_einsammeln.py` liest alle PDF-Dateien eines Ordners und schreibt eine CSV-Datei, die sich in Excel, Numbers oder LibreOffice öffnen lässt: eine Zeile pro Abgabe, eine Spalte pro Feld. Angekreuzte Kästchen erscheinen als `x`.

```
pip install pypdf
python3 antworten_einsammeln.py Abgaben_10b/
python3 antworten_einsammeln.py Abgaben_10b/ --aufgaben
```

Mit `-aufgaben` entsteht zusätzlich eine Textdatei, in der alle Antworten zu einer Aufgabe untereinanderstehen. So lässt sich Aufgabe für Aufgabe korrigieren, was den Maßstab über die ganze Klasse gleich hält. Ein Ausschnitt mit zwei Testabgaben:

```
===== a8-c =====
LK_Lena_Mueller.pdf:
    2.5
LK_Tom_Schulze.pdf:
    2,5
```

Bewertet wird weiterhin von der Lehrkraft. Das Skript sortiert nur.

8 Tipps und Stolpersteine

Alles in diesem Kapitel wurde ausprobiert.

- **Felder in Tabellen werden zu hoch.** Ein nacktes `\TextField` oder `\CheckBox` übernimmt die Höhe der Tabellenzeile. Lösung: das Feld in `\raisebox` oder `\hbox` packen, wie es die Kurzbefehle tun.
- **Optionsfelder funktionieren nicht.** Fehlt in der Präambel die Zeile `\hypersetup{unicode=false}`, erzeugt `hyperref` 7.01h Werte wie `\376\377\000z\0001` statt `z1`. Chrome speichert die Auswahl dann zwar, aber genau als diesen Zeichensalat, und beim Einsammeln (Kapitel 7) lässt sich nicht mehr ablesen, welche Option gewählt wurde.
- **Unterstriche verschwinden aus Feldnamen.** Bindestriche verwenden.
- **`\Reset` leert alle Felder**, nicht nur die in der Nähe des Knopfes, auch über mehrere `Form`-Umgebungen hinweg. In einer Leistungskontrolle hat so ein Knopf nichts verloren.
- **Lösungen nie ins Schülerdokument**, auch nicht weiß auf weiß oder auf einer ausgeblendeten Ebene. Text in einem PDF lässt sich immer herauskopieren.

- **Mehrere `Form`-Umgebungen** in einem Dokument sind möglich, eine einzige um den ganzen Inhalt ist aber übersichtlicher.
- **Platz einplanen:** Was über ein mehrzeiliges Feld hinausgeht, ist am Bildschirm über Scrollen erreichbar, auf dem Ausdruck aber abgeschnitten.

9 Befehlsreferenz

9.1 Kurzbefehle aus der newcommands.tex

Befehl	Wirkung
`\feld[B]{Name}`	einzeiliges Textfeld auf der Grundlinie, Breite B (Standard 2,4 cm)
`\zeichen{Name}`	Feld für genau ein Zeichen, zentriert, 0,8 cm breit
`\kreuz{Name}`	Ankreuzkästchen, 0,42 cm
`\antwort[H]{Name}`	mehrzeiliges Feld über die ganze Breite, Höhe H (Standard 3,2 cm)

9.2 Befehle und Optionen von hyperref

Befehl / Option	Wirkung
`\begin{Form}`	Umgebung, in der alle Felder stehen
`\TextField[...]{Text}`	Textfeld, `Text` wird als Beschriftung davorgesetzt
`\CheckBox[...]{Text}`	Ankreuzkästchen
`\ChoiceMenu[radio,...] {}{a=1,b=2}`	Optionsfelder, `Beschriftung=Wert`
`\ChoiceMenu[combo, popdown,...]{}{E,V,A}`	Auswahlliste
`\Reset{Text}`	Knopf, der alle Felder leert
`name=`	Feldname, im Dokument eindeutig, ohne Unterstrich
`width=`, `height=`	Breite und Höhe
`charsize=`	Schriftgröße der Eingabe, z. B. `11pt`
`value=`	vorgetragener Text
`default=`	Startwert, bei `radio` leer lassen: `default=`
`multiline=true`	mehrzeiliges Textfeld
`maxlen=`	höchstens so viele Zeichen
`align=`	0 links, 1 zentriert, 2 rechts
`bordercolor=`, `backgroundcolor=`	Rand- und Hintergrundfarbe als `{r g b}`, Werte 0 bis 1
`print`	Feld wird mitgedruckt

Die vollständige Liste steht in der Dokumentation von `hyperref` (`texdoc hyperref`, Abschnitt „Forms“).

10 Zusammenfassung

- Formularfelder erzeugt `hyperref`, alle Felder stehen in `\begin{Form} ... \end{Form}`, dazu `\hypersetup{unicode=false}`.
- Für Arbeitsblätter und Leistungskontrollen reichen vier Kurzbefehle: `\feld`, `\zeichen`, `\kreuz` und `\antwort`.
- Jedes Feld braucht einen eindeutigen Namen ohne Unterstrich, am besten nach Aufgaben-

nummer.

- Eine Papierfassung wird digital, indem man Kästchen, Lücken und Linien durch Felder ersetzt und die Lösungen herausnimmt.
- Vor dem ersten Einsatz eine Probeabgabe auf den Geräten der Klasse machen.

Für den Unterricht heißt das: Dieselbe Datei, aus der die Papierfassung entsteht, liefert mit wenigen Änderungen eine digitale Fassung. Die Schülerinnen und Schüler brauchen nur ein PDF-Programm, die Lehrkraft korrigiert so individuell wie auf Papier und hat die Abgaben trotzdem geordnet auf der Lernplattform.